

My Project

Generated by Doxygen 1.9.1

1 Todo List	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 NewtonSolver< N, M > Class Template Reference	7
4.1.1 Detailed Description	9
4.1.2 Member Function Documentation	9
4.1.2.1 armijo_search()	9
4.1.2.2 assign_AD_vector()	9
4.1.2.3 assign_F()	10
4.1.2.4 get_ iterations()	10
4.1.2.5 get_success()	10
4.1.2.6 Jacobian()	11
4.1.2.7 merit_fun()	11
4.1.2.8 set_settings()	11
4.2 NewtonSolverSettings Struct Reference	12
4.2.1 Detailed Description	12
4.2.2 Member Data Documentation	12
4.2.2.1 beta	12
4.2.2.2 gamma	12
4.2.2.3 max_ iterations	13
4.2.2.4 t_max	13
4.2.2.5 tol	13
5 File Documentation	15
5.1 src/AD_example.cpp File Reference	15
5.1.1 Detailed Description	16
5.1.2 Function Documentation	16
5.1.2.1 HelicalValley()	16
Index	17

Chapter 1

Todo List

Member `NewtonSolver< N, M >::set_settings` (int max_iterations, double tol)

Also add possibility of setting Armijo line search parameters.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

NewtonSolver< N, M >	
Newton solver for systems of nonlinear equations	7
NewtonSolverSettings	12

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/ Newton_solver.hh	??
src/ AD_example.cpp Example of root finding for nonlinear systems with the Newton solver	15

Chapter 4

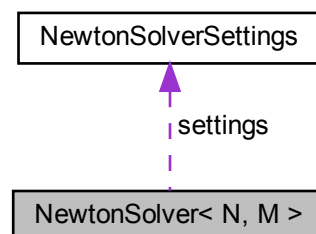
Class Documentation

4.1 NewtonSolver< N, M > Class Template Reference

Newton solver for systems of nonlinear equations.

```
#include <Newton_solver.hh>
```

Collaboration diagram for NewtonSolver< N, M >:



Public Types

- using **AD_N** = TinyAD::Double< N >
AD scalar type with N-dimensional gradient. It is the type that take into account the variables respect to which we differentiate.
- using **FuncSig** = void(*) (const Eigen::Matrix< **AD_N**, N, 1 > &, Eigen::Matrix< **AD_N**, N, 1 > &)
It is the signature of the function that computes the system of equations to be differentiated. It takes as input an Eigen vector of AD scalars, and outputs an Eigen vector of AD scalars, which is the evaluation of the system of equations at the input point. The user has to provide a function with this signature to compute the system of equations to be solved.
- using **FuncSigDouble** = void(*) (const Eigen::Matrix< double, N, 1 > &, Eigen::Matrix< double, N, 1 > &)
Signature for the same function as above, but without automatic differentiation. Used when just an evaluation of the function is needed.

Public Member Functions

- bool [get_success](#) () const
Returns whether the solver converged successfully.
- int [get_iterations](#) () const
Returns the number of iterations performed by the solver.
- void [set_settings](#) (int max_iterations, double tol)
Returns the number of iterations performed by the solver.
- [NewtonSolverSettings](#) [get_settings](#) () const
Returns the settings struct.
- void [set_initial_guess](#) (Eigen::Matrix< double, N, 1 > &guess)
Provide an initial guess to the solver.
- void [print_initial_guess](#) () const
- void [Jacobian](#) (const Eigen::Matrix< [AD_N](#), N, 1 > &input, [FuncSig](#) func)
Function to compute Jacobian matrix with automatic differentiation.
- const Eigen::Ref< const Eigen::MatrixXd > [get_solution](#) () const
Returns the solution found by the solver.
- void [solve](#) ([FuncSig](#) func, [FuncSigDouble](#) func_double)
Solve the problem, given the function to be solved, coherent in size with the instantiated solver.

Protected Member Functions

- void [assign_F](#) (const Eigen::Matrix< [AD_N](#), M, 1 > &F_active)
Function to store the value of an eigen vector of AD scalars into the attribute `F_val` that stores the value of the system of equations.
- template<int Size>
void [assign_AD_vector](#) (const Eigen::Matrix< [AD_N](#), Size, 1 > &vec_source, Eigen::Matrix< double, Size, 1 > *&vec_dest)
Function to store the value of an eigen vector of AD scalars into an eigen vector of doubles. It is templetized on the size of the vectors.
- void [assign_solution](#) (const Eigen::Matrix< [AD_N](#), N, 1 > &sol)
Function to store the value of an eigen vector of AD scalars into the solution attribute, which is an eigen vector of doubles.
- double [merit_fun](#) (const Eigen::Matrix< double, N, 1 > &x, Eigen::Matrix< double, M, 1 > *&tmp, [FuncSigDouble](#) func)
Computes the merit function value: $\phi(x) = 0.5 \cdot \text{norm}_2(F(x))^2$.
- double [armijo_search](#) (Eigen::Matrix< double, N, 1 > &x, Eigen::Vector< double, N > &d, [FuncSigDouble](#) func, Eigen::Vector< double, N > &f_val_x)
Performs Armijo line search to find a suitable step size t that satisfies the Armijo condition.

Protected Attributes

- [NewtonSolverSettings](#) **settings**
- bool **success** = false
- int **iterations** = 0
- Eigen::Vector< double, N > **solution** = Eigen::Matrix<double, N, 1>::Zero()
- Eigen::Matrix< double, N, 1 > * **initial_guess** = new Eigen::Matrix<double, N, 1>(Eigen::Matrix<double, N, 1>::Zero())
- Eigen::Matrix< double, M, N > * **J** = nullptr
- Eigen::Matrix< double, M, 1 > * **F_val** = nullptr
- Eigen::Matrix< [AD_N](#), M, 1 > * **F_val_ad** = nullptr
- Eigen::Matrix< double, N, 1 > **d** = Eigen::Matrix<double, N, 1>::Zero()
- Eigen::Matrix< double, M, 1 > * **lhs_armijo** = nullptr
- Eigen::Matrix< double, M, 1 > * **rhs_armijo** = nullptr

4.1.1 Detailed Description

```
template<int N, int M>
class NewtonSolver< N, M >
```

Newton solver for systems of nonlinear equations.

It is a class that exploits automatic differentiation to compute the Jacobian matrix of the system of equations at each iteration, and uses it to compute the descent direction, employing Armijo line search. It is templated on the number of variables and equations, and it requires the user to provide the function that computes the system of equations to be differentiated.

Template Parameters

<i>N</i>	number of variables
<i>M</i>	number of equations

4.1.2 Member Function Documentation

4.1.2.1 armijo_search()

```
template<int N, int M>
double NewtonSolver< N, M >::armijo_search (
    Eigen::Matrix< double, N, 1 > & x,
    Eigen::Vector< double, N > & d,
    FuncSigDouble func,
    Eigen::Vector< double, N > & f_val_x ) [inline], [protected]
```

Performs Armijo line search to find a suitable step size *t* that satisfies the Armijo condition.

Parameters

<i>x</i>	The current point in the optimization.
<i>d</i>	The descent direction found with the Jacobian and the resolution of the LU system.
<i>func</i>	The function for which to perform the line search.
<i>f_val_x</i>	The value of the function at the current point <i>x</i> , used to compute the Armijo condition.

Returns

The step size *t* that satisfies the Armijo condition.

4.1.2.2 assign_AD_vector()

```
template<int N, int M>
template<int Size>
```

```
void NewtonSolver< N, M >::assign_AD_vector (
    const Eigen::Matrix< AD_N, Size, 1 > & vec_source,
    Eigen::Matrix< double, Size, 1 > *& vec_dest ) [inline], [protected]
```

Function to store the value of an eigen vector of AD scalars into an eigen vector of doubles. It is templetized on the size of the vectors.

Parameters

<i>vec_source</i>	The eigen vector of AD scalars to be stored in <i>vec_dest</i> .
<i>vec_dest</i>	The eigen vector of doubles where the values will be stored.

4.1.2.3 assign_F()

```
template<int N, int M>
void NewtonSolver< N, M >::assign_F (
    const Eigen::Matrix< AD_N, M, 1 > & F_active ) [inline], [protected]
```

Function to store the value of an eigen vector of AD scalars into the attribute *F_val* that stores the value of the system of equations.

Parameters

<i>F_active</i>	The eigen vector of AD scalars to be stored in <i>F_val</i> .
-----------------	---

4.1.2.4 get_iterations()

```
template<int N, int M>
int NewtonSolver< N, M >::get_iterations ( ) const [inline]
```

Returns the number of iterations performed by the solver.

Returns

The number of iterations performed.

4.1.2.5 get_success()

```
template<int N, int M>
bool NewtonSolver< N, M >::get_success ( ) const [inline]
```

Returns whether the solver converged successfully.

Returns

true if the solver converged, false otherwise.

4.1.2.6 Jacobian()

```
template<int N, int M>
void NewtonSolver< N, M >::Jacobian (
    const Eigen::Matrix< AD_N, N, 1 > & input,
    FuncSig func ) [inline]
```

Function to compute Jacobian matrix with automatic differentiation.

It does not have return but it assigns the values of the Jacobian to the J attribute. It also keep track of the value of the function at the current input point, which is stored in F_val_ad.

Parameters

<i>input</i>	The point at which to compute the Jacobian, it must be an eigen vector of AD scalars.
<i>func</i>	The function that computes the system of equations. It must have the signature FuncSig.

4.1.2.7 merit_fun()

```
template<int N, int M>
double NewtonSolver< N, M >::merit_fun (
    const Eigen::Matrix< double, N, 1 > & x,
    Eigen::Matrix< double, M, 1 > *& tmp,
    FuncSigDouble func ) [inline], [protected]
```

Computes the merit function value: $\phi(x) = 0.5 \cdot \text{norm}_2(F(x))^2$.

Parameters

<i>x</i>	The point at which to evaluate the merit function.
<i>tmp</i>	Temporary vector for intermediate computations.
<i>func</i>	The function for which to compute the merit function.

Returns

The value of the merit function.

4.1.2.8 set_settings()

```
template<int N, int M>
void NewtonSolver< N, M >::set_settings (
    int max_iterations,
    double tol ) [inline]
```

Returns the number of iterations performed by the solver.

Returns

The number of iterations performed.

Todo Also add possibility of setting Armijo line search parameters.

The documentation for this class was generated from the following file:

- include/Newton_solver.hh

4.2 NewtonSolverSettings Struct Reference

```
#include <Newton_solver.hh>
```

Public Attributes

- int `max_iterations` = 1000
- double `tol` = 1e-9
- double `gamma` = 1e-4
- double `beta` = 0.5
- double `t_max` = 1.0

4.2.1 Detailed Description

Simple struct to hold solver settings

4.2.2 Member Data Documentation

4.2.2.1 `beta`

```
double NewtonSolverSettings::beta = 0.5
```

Beta for Armijo line search, parameters that multiplies t to update it at each iteration during line search

4.2.2.2 `gamma`

```
double NewtonSolverSettings::gamma = 1e-4
```

Gamma for Armijo line search

4.2.2.3 max_iterations

```
int NewtonSolverSettings::max_iterations = 1000
```

Maximum number of iterations

4.2.2.4 t_max

```
double NewtonSolverSettings::t_max = 1.0
```

Maximum step size and initial value for Armijo line search

4.2.2.5 tol

```
double NewtonSolverSettings::tol = 1e-9
```

Tolerance for convergence

The documentation for this struct was generated from the following file:

- include/Newton_solver.hh

Chapter 5

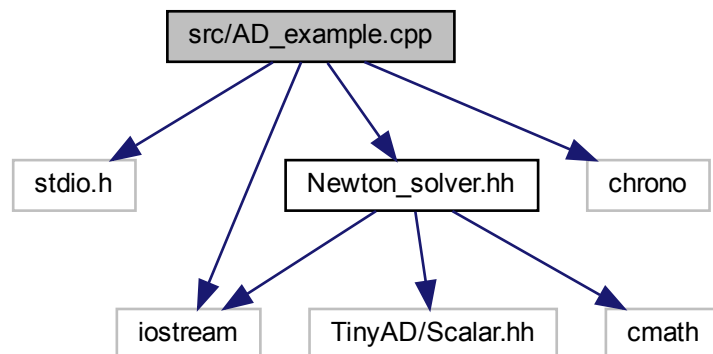
File Documentation

5.1 src/AD_example.cpp File Reference

Example of root finding for nonlinear systems with the Newton solver.

```
#include <stdio.h>
#include <iostream>
#include "Newton_solver.hh"
#include <chrono>
```

Include dependency graph for AD_example.cpp:



Functions

- `template<typename Scalar >`
`void HelicalValley (const Eigen::Matrix< Scalar, 3, 1 > &v, Eigen::Matrix< Scalar, 3, 1 > &out)`
Helical Valley function, a common test problem for optimization algorithms. Every systems of equation has to be described by a function of this form, that is passes as argument to the solver.
- `int main ()`

Variables

- auto **start** = std::chrono::high_resolution_clock::now()

5.1.1 Detailed Description

Example of root finding for nonlinear systems with the Newton solver.

5.1.2 Function Documentation

5.1.2.1 HelicalValley()

```
template<typename Scalar >
void HelicalValley (
    const Eigen::Matrix< Scalar, 3, 1 > & v,
    Eigen::Matrix< Scalar, 3, 1 > & out )
```

Helical Valley function, a common test problem for optimization algorithms. Every systems of equation has to be described by a function of this form, that is passes as argument to the solver.

Template Parameters

<i>Scalar</i>	The scalar type of the input and output vectors. The solver requires double and double AD scalar type, the type used by TinyAD.
---------------	---

Parameters

<i>v</i>	The input vector representing the point at which to evaluate the function.
<i>out</i>	The output vector where the function value will be stored.

Index

AD_example.cpp
 HelicalValley, [16](#)
armijo_search
 NewtonSolver< N, M >, [9](#)
assign_AD_vector
 NewtonSolver< N, M >, [9](#)
assign_F
 NewtonSolver< N, M >, [10](#)

beta
 NewtonSolverSettings, [12](#)

gamma
 NewtonSolverSettings, [12](#)
get_iterations
 NewtonSolver< N, M >, [10](#)
get_success
 NewtonSolver< N, M >, [10](#)

HelicalValley
 AD_example.cpp, [16](#)

Jacobian
 NewtonSolver< N, M >, [10](#)

max_iterations
 NewtonSolverSettings, [12](#)
merit_fun
 NewtonSolver< N, M >, [11](#)

NewtonSolver< N, M >, [7](#)
 armijo_search, [9](#)
 assign_AD_vector, [9](#)
 assign_F, [10](#)
 get_iterations, [10](#)
 get_success, [10](#)
 Jacobian, [10](#)
 merit_fun, [11](#)
 set_settings, [11](#)
NewtonSolverSettings, [12](#)
 beta, [12](#)
 gamma, [12](#)
 max_iterations, [12](#)
 t_max, [13](#)
 tol, [13](#)

set_settings
 NewtonSolver< N, M >, [11](#)
src/AD_example.cpp, [15](#)

t_max
 NewtonSolverSettings, [13](#)
tol
 NewtonSolverSettings, [13](#)